MITM Attack using ARP Spoofing & Bettercap

BY : ALAY PATEL

1. Abstract

This project explores the fundamental vulnerabilities within the Address Resolution Protocol (ARP) and demonstrates how these weaknesses can be exploited to perform a Man-in-the-Middle (MITM) attack. By utilizing tools like arpspoof, Wireshark, and the Bettercap framework, we successfully intercepted sensitive data, including login credentials, from a target machine. The report concludes with a detailed analysis of the risks associated with unencrypted protocols and provides mitigation strategies for network administrators.

2. Theoretical Background

2.1 The Role of ARP in Networking

On a Local Area Network (LAN), devices communicate using MAC (Media Access Control) addresses. However, applications use IP addresses. ARP acts as the bridge:

- When Host A wants to talk to Host B, it broadcasts an **ARP Request**: *"Who has IP 192.168.1.5?"*
- Host B sends an **ARP Reply**: *"I have that IP, and my MAC is AA:BB:CC..."*
- Host A stores this in an **ARP Cache** to remember the connection.

2.2 The Vulnerability: ARP Poisoning

ARP was designed in an era where internal networks were considered "trusted." It lacks authentication. A device will accept an ARP Reply even if it never sent a Request. An attacker can send "Gratuitous ARP" messages to the victim and the gateway, claiming to be the other party. This "poisons" the ARP cache of both devices.

3. Environment Architecture

To conduct this research safely and ethically, a virtualized lab environment was established:

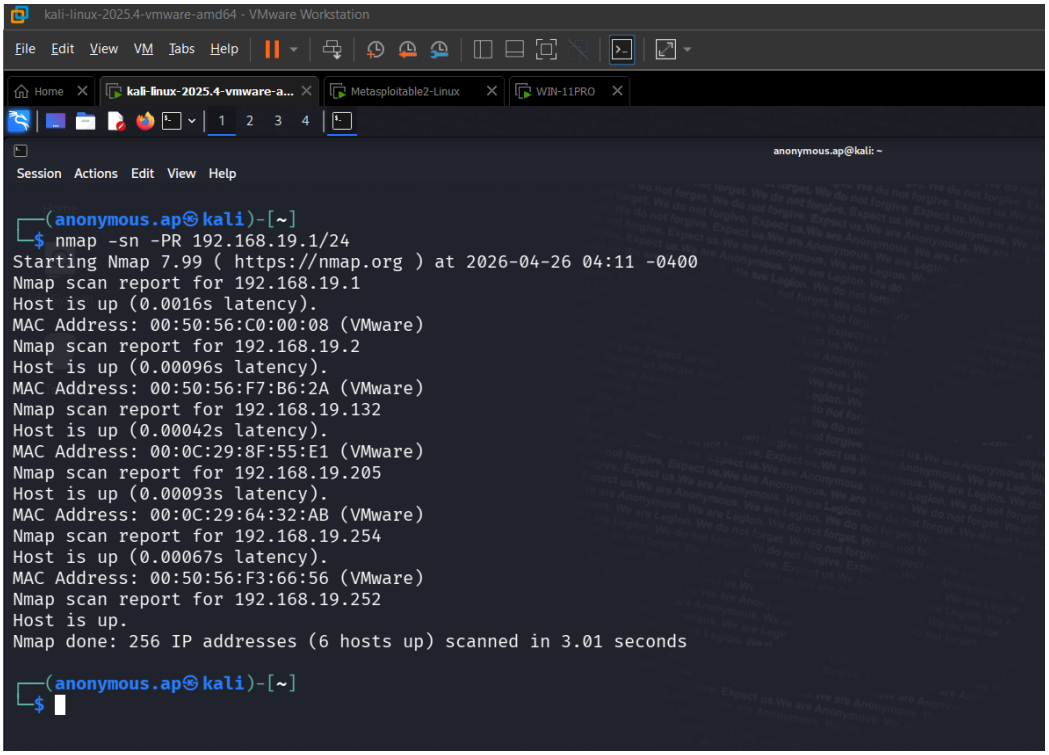
- **Host OS:** Windows 11 Pro
- **Attacker Machine:** Kali Linux (Rolling Edition)
- **Target Machine 1 (Victim):** Windows 11 Pro (Virtual Machine)
- **Target Machine 2 (Server):** Metasploitable 2 (A deliberately vulnerable Linux VM hosting FTP and HTTP services).
- **Network Config:** All machines set to a "NAT Network" or "Bridged" mode to simulate a real-world LAN.

4. Execution Methodology (The Manual Process)

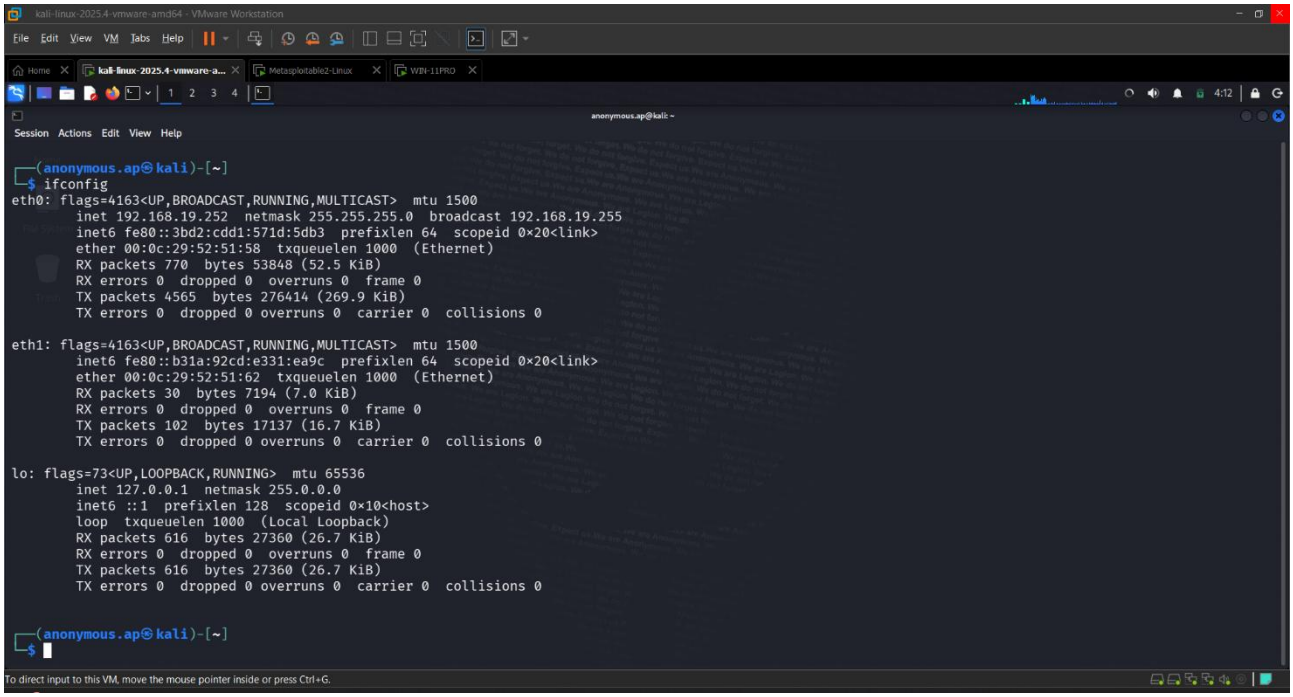
Phase I: Reconnaissance and Discovery

We cannot attack what we cannot see. Using a network discovery tool (like a Python-based scanner or Nmap), we mapped the network:

- Attacker (Kali): 192.168.19.252
- Victim (Windows): 192.168.19.132
- Metasploit 2 : 192.168.19.205
- Gateway (Router): 192.168.19.2



Nmap network scan using the command nmap -sn -PR 192.168.19.1/24. This specific scan (ARP ping scan) was used to identify active hosts on the local network without performing a full port scan.



It highlights your active IP address as **192.168.19.252** on interface **eth0**, confirming your machine is connected to the same subnet you previously scanned.

Phase II: The Two-Way Spoof

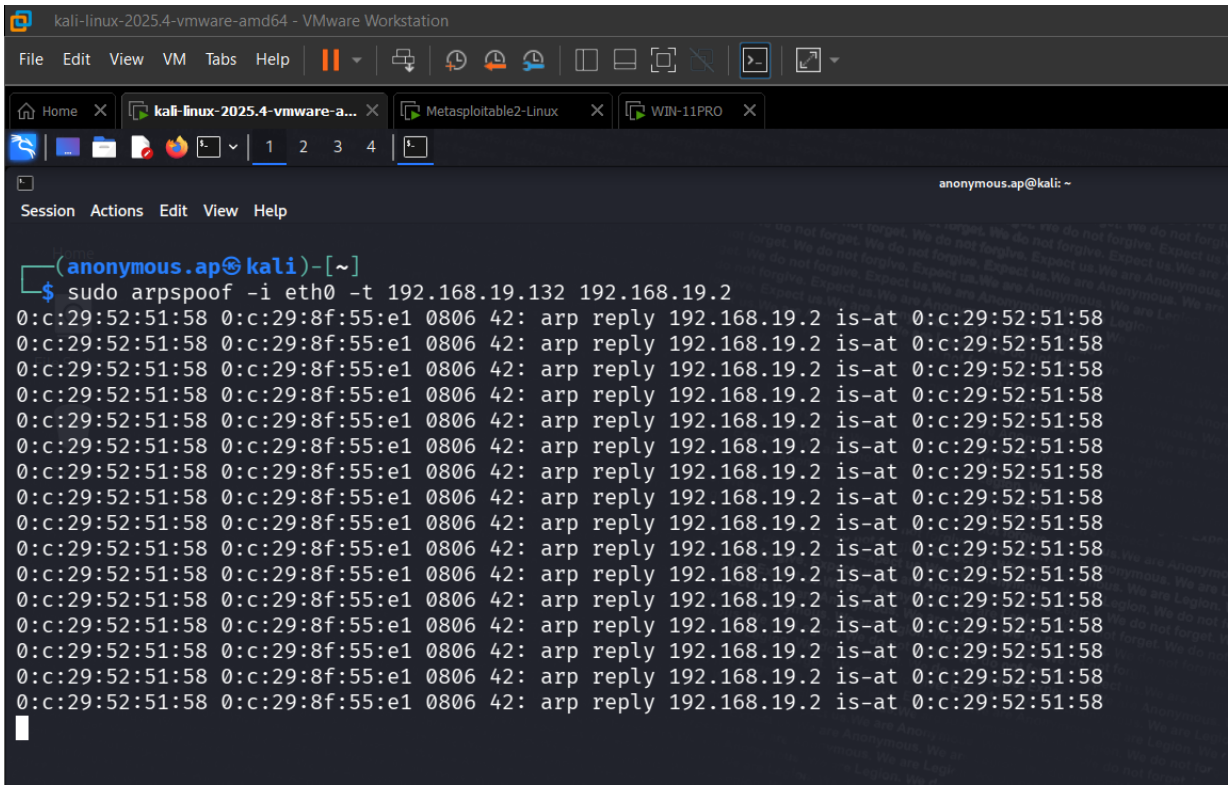
We execute the arpspoof command in two separate terminal windows:

1. **Targeting the Victim:** arpspoof -i eth0 -t 192.168.19.132 192.168.19.2

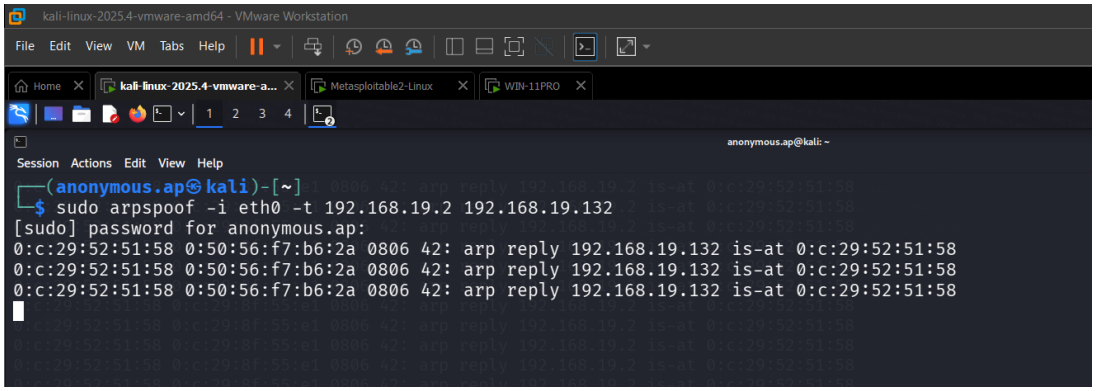
(Telling the victim: "I am the router")

2. **Targeting the Router:** arpspoof -i eth0 -t 192.168.19.2 192.168.19.132

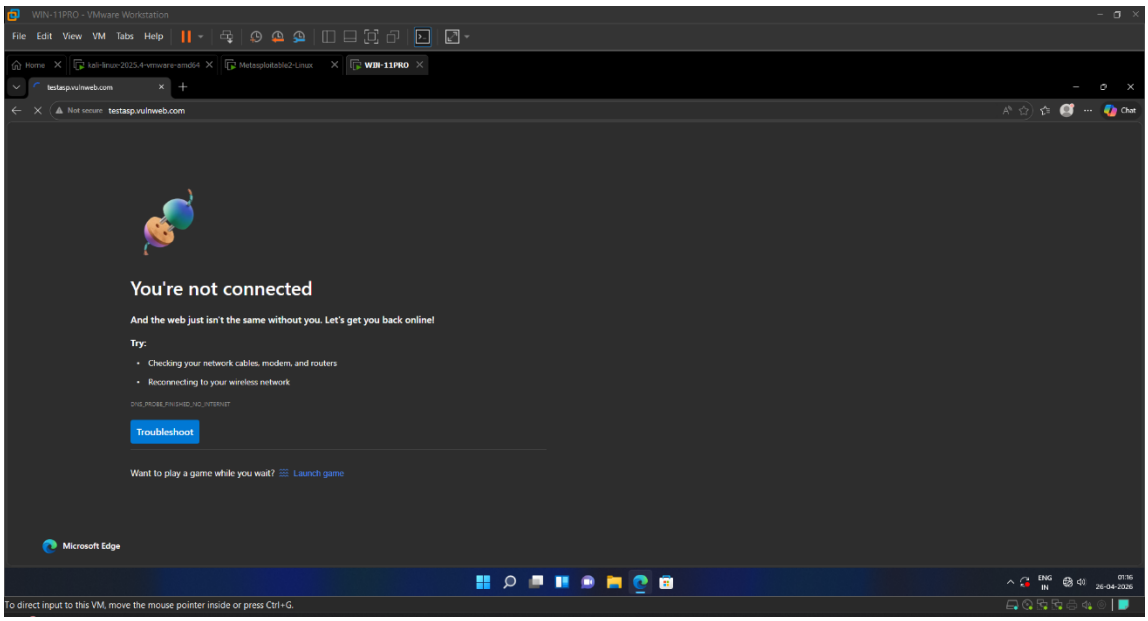
(Telling the router: "I am the victim")



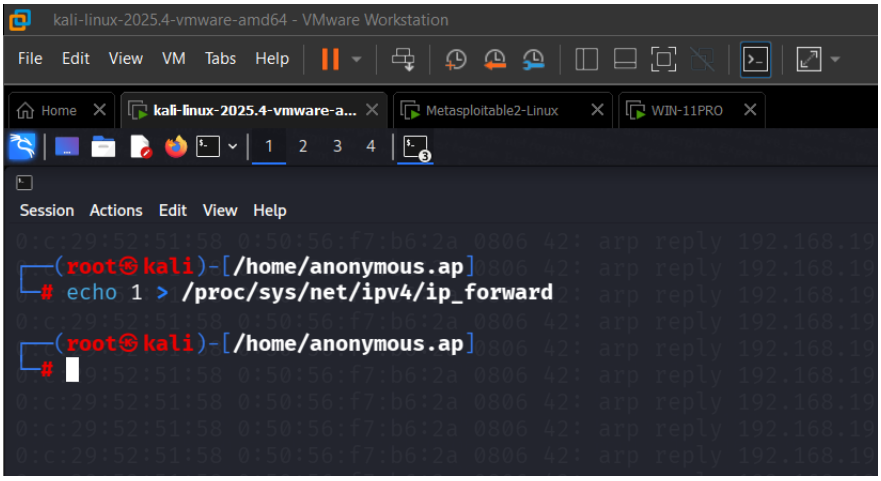
I am currently targeting the host at **192.168.19.132** (my Metasploitable2 VM) and deceiving it into believing that my Kali machine is the gateway at **192.168.19.2**. By sending a continuous stream of forged ARP reply packets, I am poisoning the victim's ARP cache to redirect its traffic through my machine.



I have initiated the second half of the ARP poisoning attack by targeting the gateway at **192.168.19.2** and telling it that the victim at **192.168.19.132** is located at my MAC address.
By running this alongside my previous command, I am now poisoning both sides of the communication channel. This ensures a complete **bi-directional intercept**, allowing me to capture and monitor the full conversation between the target VM and the rest of the network.



The reason there is no internet in this screenshot is that while I have successfully poisoned the ARP cache of my target machine, my Kali Linux machine is currently **dropping** all the traffic it receives.

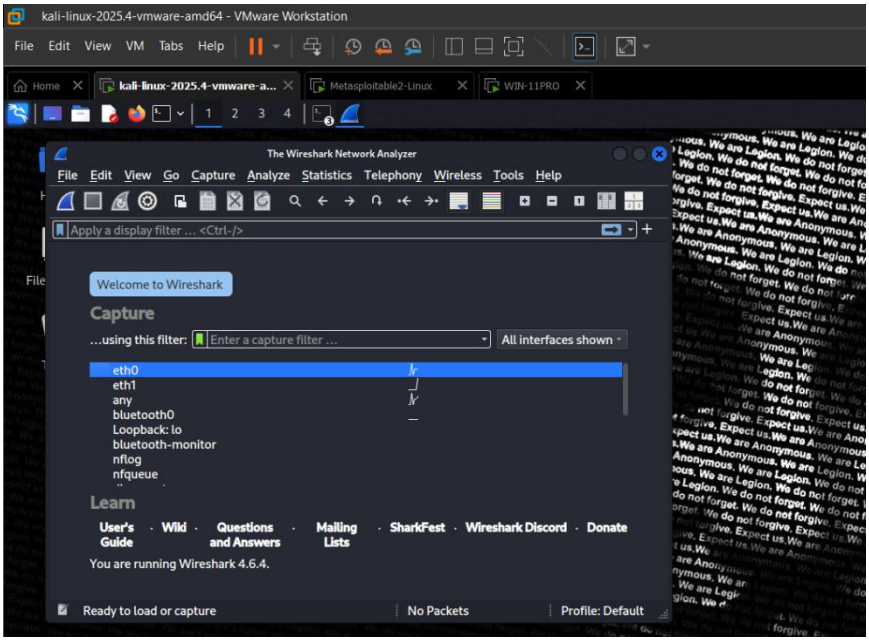


I am currently enabling **IP forwarding** on my Kali Linux machine to resolve the connectivity issue on the target system. By executing the command *echo 1 > /proc/sys/net/ipv4/ip_forward,* I am instructing my operating system to act as a router.

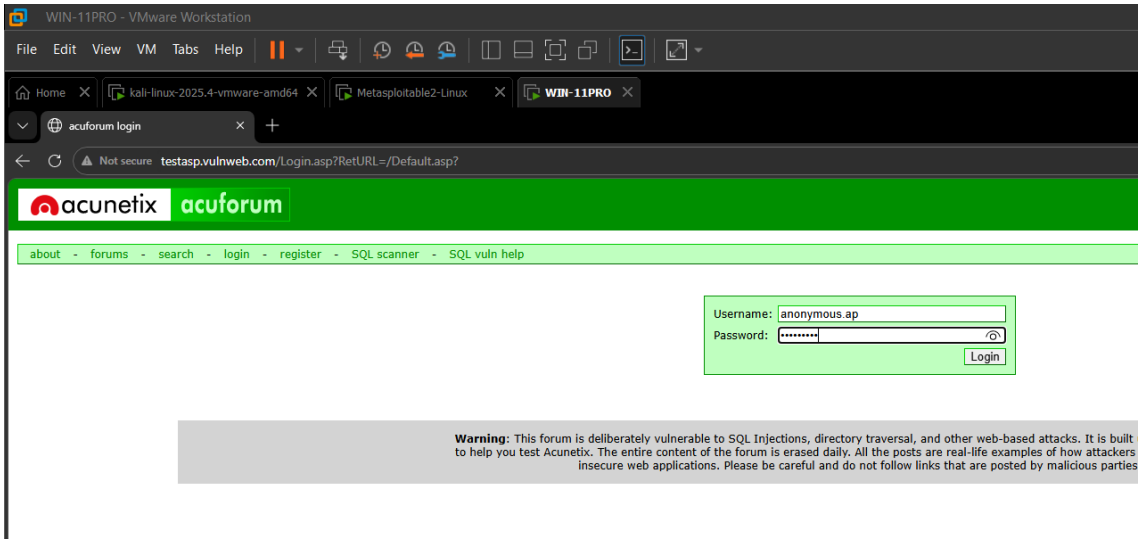
Phase IV: Packet Sniffing with Wireshark

With the traffic flowing through our Kali machine, we launch Wireshark.

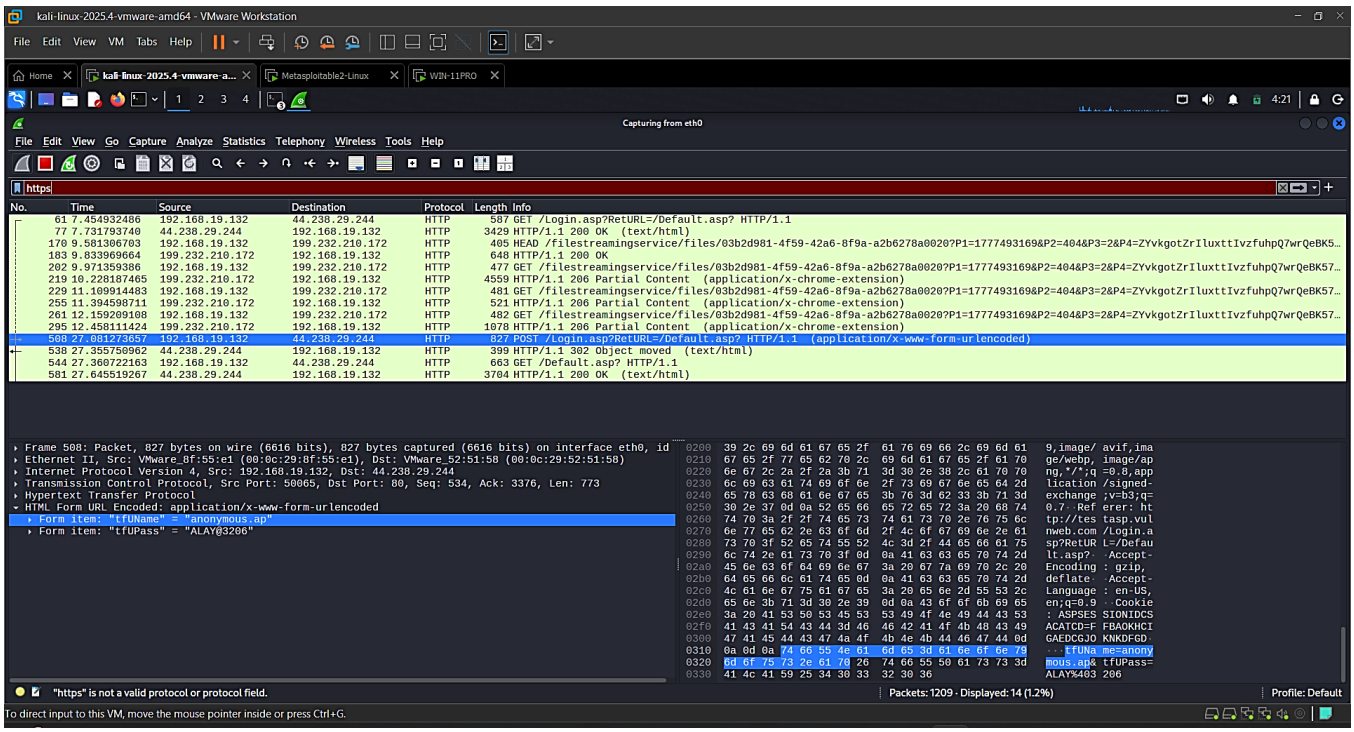
- **Filter applied:** ftp or http
- **Observation**: When the victim logged into the Metasploitable server via FTP, Wireshark captured the USER and PASS commands in cleartext.



Now, I have started Wireshark for packet sniffing

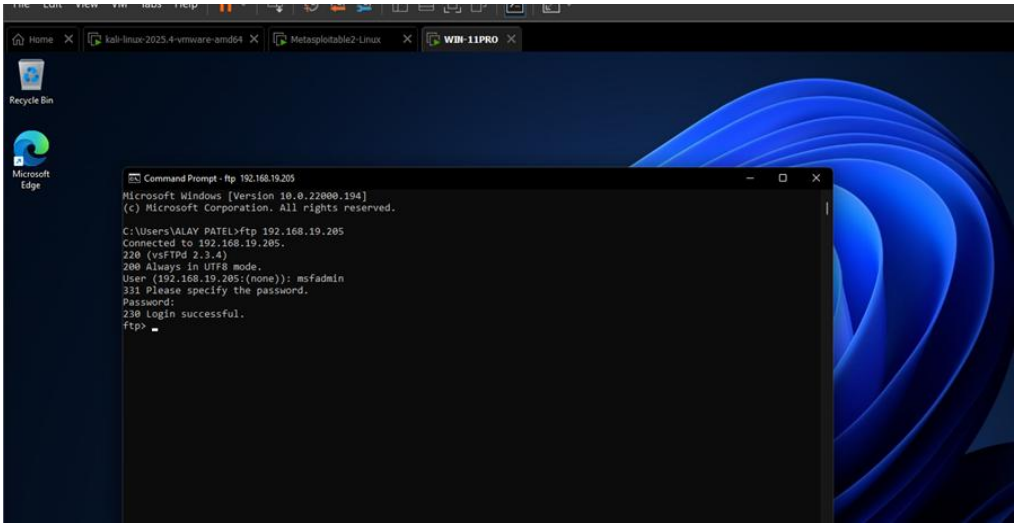


I have entered the credentials anonymous.ap into the login fields on the **WIN-11PRO** VM. Since this traffic is being routed through my Kali machine via the active ARP spoofing attack, these plain-text credentials are now ready to be intercepted and captured using a packet sniffer.

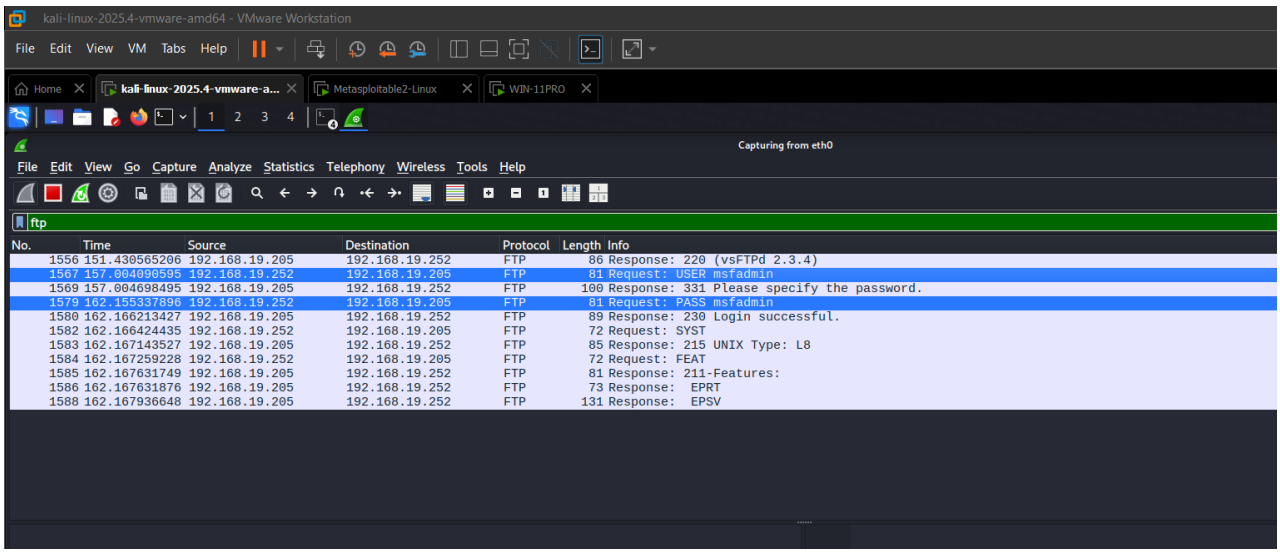


I have successfully completed the attack by using **Wireshark** to sniff the redirected traffic. By filtering for HTTP traffic, I captured the **POST request** sent from the victim's machine (192.168.19.132).
Inside the packet data, I can clearly see the intercepted credentials in plain text:

- **tfUName:** anonymous.ap
- **tfUPass:** ALAY@3206



I am currently establishing an **FTP connection** to the target machine at **192.168.19.205**. After connecting to the vsFTPd 2.3.4 service, I successfully authenticated using the credentials msfadmin for both the username and password.



I am using **Wireshark** to capture and analyze an active **FTP session** between my Kali machine and the target at 192.168.19.205. By filtering for the FTP protocol, I have successfully intercepted the authentication handshake in plain text.
The capture clearly shows the unencrypted transmission of credentials:

- **Request: USER msfadmin**
- **Request: PASS msfadmin**

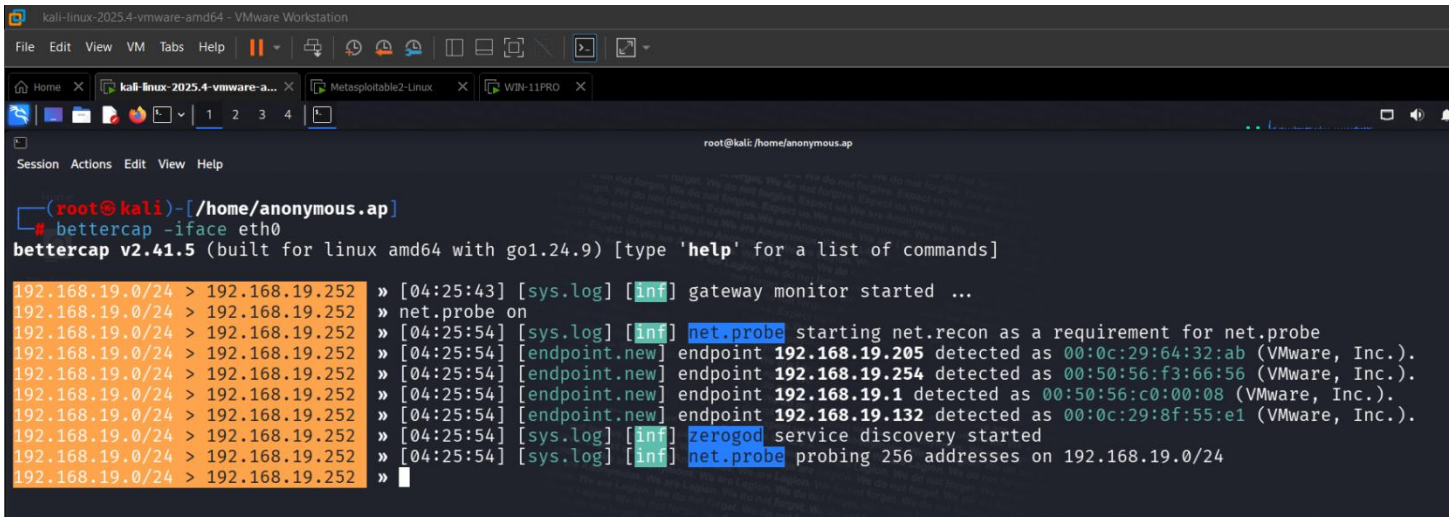
5. Advanced Execution (Bettercap Framework)

Bettercap is the modern successor to older tools like Ettercap. It provides a modular, automated approach to MITM attacks.

Workflow in Bettercap:

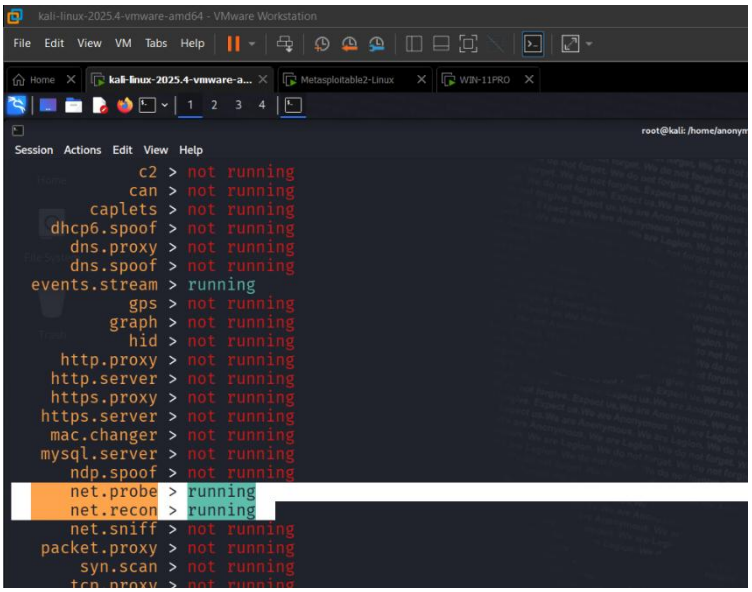
1. Initialization: bettercap -iface eth0
2. Probe the Network: net.probe on (Sends dummy packets to discover all active hosts).
3. Establish Targets: set arp.spoof.targets 192.168.80.132
4. Full Duplex Spoofing: set arp.spoof.fullduplex true (Spoofs both target and gateway simultaneously).
5. Activation: arp.spoof on
6. Real-time Sniffing: net.sniff on

Bettercap Advantage: Unlike Wireshark, which shows thousands of packets, Bettercap's net.sniff module is "intelligent." It recognizes authentication strings for FTP, HTTP, and MQTT, and prints them directly to the console in a readable format.



I am currently utilizing **Bettercap** on the eth0 interface to perform advanced network reconnaissance and monitoring. By enabling the **net.probe** module, I have initiated a scan across the 192.168.19.0/24 subnet to automatically discover active endpoints.
The tool has already identified several live hosts, including:

- **192.168.19.205** (Metasploitable2)
- **192.168.19.132** (Victim VM)
- **192.168.19.1** (Gateway)



I have reviewed the status of the various modules within **Bettercap** to manage my network attack surface. By checking the running services, I can confirm that **net.probe** and **net.recon** are active, which ensures continuous discovery and mapping of all hosts on the local subnet.

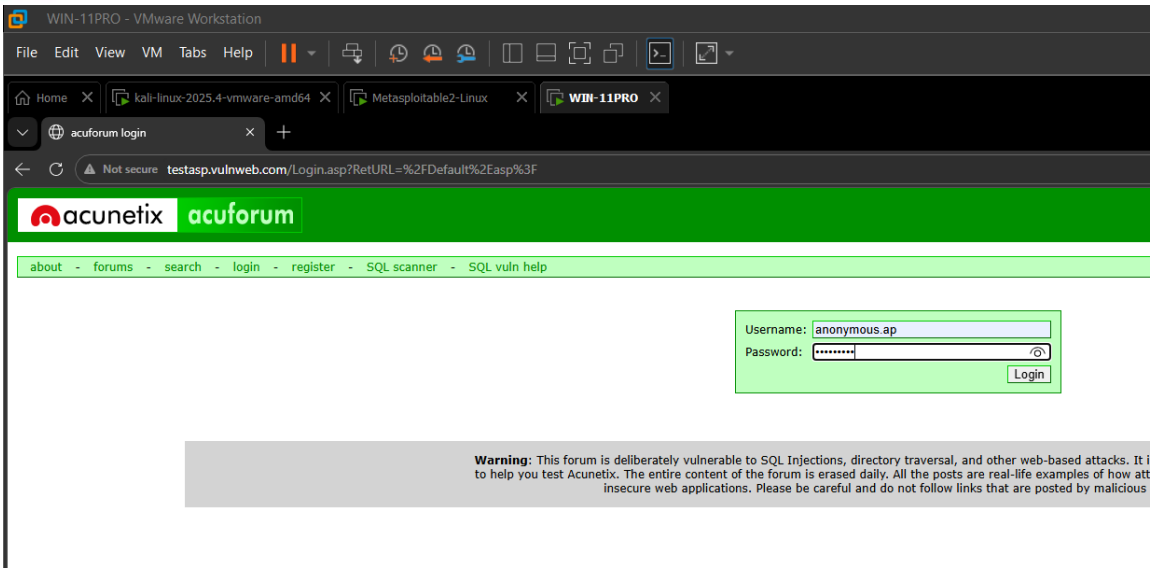
```
192.168.19.0/24 > 192.168.19.252 » help arp.spoof

arp.spoof (not running): Keep spoofing selected hosts on the network.

   arp.spoof on : Start ARP spoofer.
     arp.ban on : Start ARP spoofer in ban mode, meaning the target(s) connectivity will not work.
  arp.spoof off : Stop ARP spoofer.
    arp.ban off : Stop ARP spoofer.

  Parameters

    arp.spoof.fullduplex : If true, both the targets and the gateway will be attacked, otherwise only the target (if the router has ARP spoofing
protections in place this will make the attack fail). (default=false)
      arp.spoof.internal : If true, local connections among computers of the network will be spoofed, otherwise only connections going to and com
ing from the external network. (default=false)
  arp.spoof.skip_restore : If set to true, targets arp cache won't be restored when spoofing is stopped. (default=false)
       arp.spoof.targets : Comma separated list of IP addresses, MAC addresses or aliases to spoof, also supports nmap style IP ranges. (default=
<entire subnet>)
     arp.spoof.whitelist : Comma separated list of IP addresses, MAC addresses or aliases to skip while spoofing. (default=)

192.168.19.0/24 > 192.168.19.252 »
```

Now I am exploring the **help menu for the arp.spoof module** within Bettercap to configure the parameters for my next attack. By reviewing the documentation, I am identifying key commands such as **arp.spoof on** to start the redirect and **arp.ban on** to perform a denial-of-service attack on specific targets.

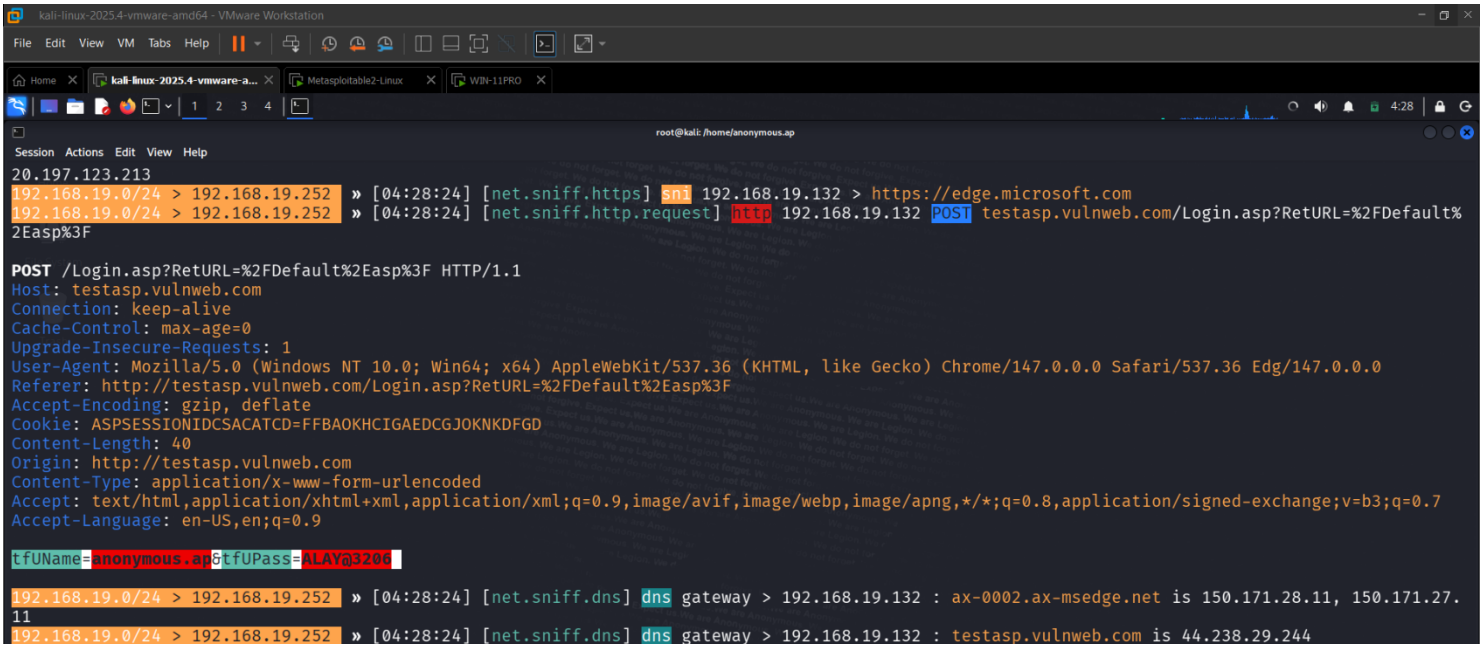
I am specifically focusing on the **arp.spoof.fullduplex** parameter to ensure both the target and the gateway are spoofed simultaneously, as well as **arp.spoof.targets** to define the specific IP addresses for interception. This preparation ensures that my **Man-in-the-Middle** setup is precisely configured for maximum efficiency within the network lab.

```
root@kali: /home/anonymous.ap
Session Actions Edit View Help
     arp.spoof.whitelist : Comma separated list of IP addresses, MAC addresses or aliases to skip while spoofing. (default=)

192.168.19.0/24 > 192.168.19.252 » set arp.spoof.fullduplex true
192.168.19.0/24 > 192.168.19.252 » set arp.spoof.targets 192.168.19.132
192.168.19.0/24 > 192.168.19.252 » net.sniff on
192.168.19.0/24 > 192.168.19.252 » help net.sniff

net.sniff (running): Sniff packets from the network.
```

I am configuring **Bettercap** to launch a more automated and efficient **Man-in-the-Middle** attack. By executing set **arp.spoof.fullduplex** true, I am ensuring that I poison both the target and the gateway for complete traffic interception, while set **arp.spoof.targets** 192.168.19.132 locks the attack onto my specific victim VM.
Additionally, I have enabled the packet sniffer using **net.sniff** on to begin capturing data in real-time directly within the terminal. I am now accessing the help **net.sniff** documentation to further refine my capture filters, allowing me to isolate sensitive information like login credentials or session cookies more effectively.

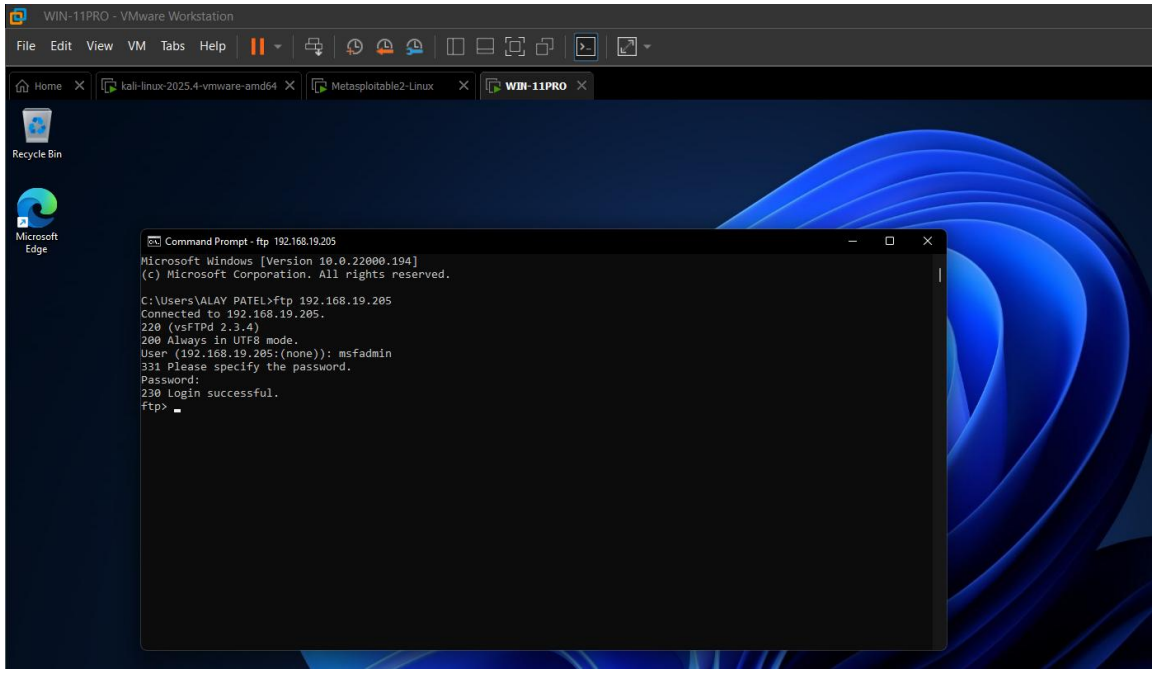


I have accessed the **WIN-11PRO** virtual machine to simulate a user login on the vulnerable **Acuforum** website. I am entering the username anonymous.ap and a password into the login form to generate network traffic.

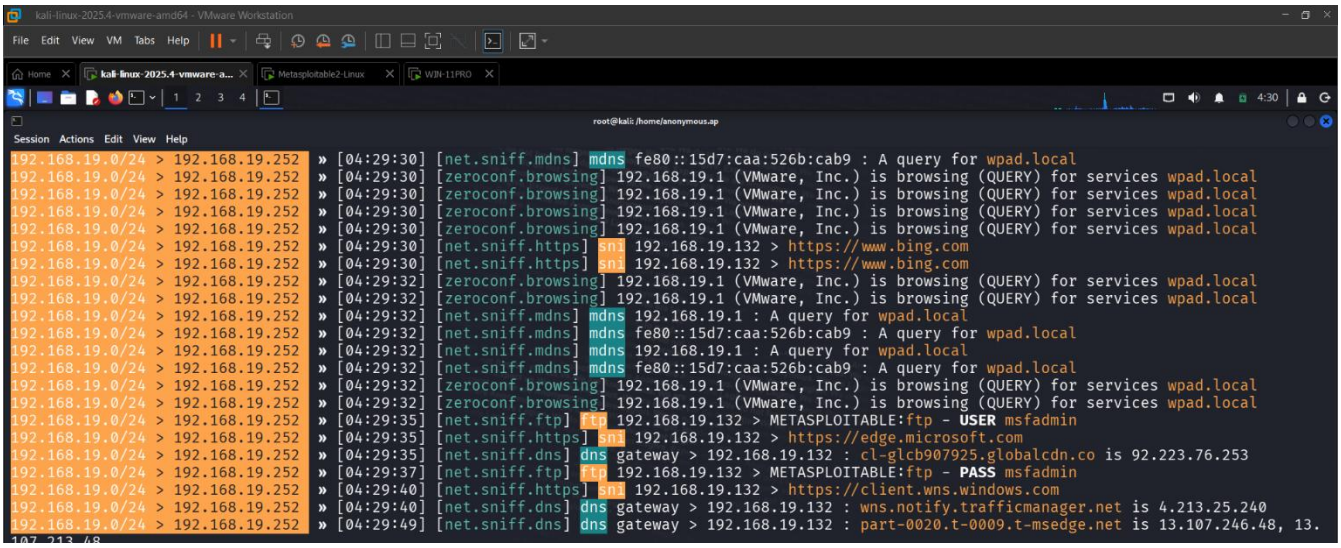


I have successfully captured the plain-text login credentials directly within the **Bettercap** terminal. By monitoring the redirected traffic from the target (192.168.19.132), the tool intercepted an **HTTP POST** request sent to the login page of the vulnerable website.
Bettercap's **net.sniff** module automatically parsed the form data, clearly highlighting the sensitive information:

- **tfUName:** anonymous.ap
- **tfUPass:** ALAY@3206



I have accessed the **Windows 11 Pro** machine and opened the Command Prompt to establish an **FTP connection** to the Metasploitable2 server at **192.168.19.205**. I successfully logged in using the default credentials msfadmin for both the username and password.



I have successfully intercepted the **FTP authentication** process in real-time using **Bettercap**. By monitoring the traffic between the Windows machine (192.168.19.132) and the Metasploitable server, the net.sniff module has automatically identified and extracted the credentials.
The terminal output clearly displays the captured plain-text data:

- **[net.sniff.ftp]** USER: msfadmin
- **[net.sniff.ftp]** PASS: msfadmin

The "Silent" Attack

One of the most important findings was that the victim's experience remained unchanged. The internet speed was stable, and no "security alerts" popped up. This highlights why ARP spoofing is a "silent killer" in network security.

---

6. Defensive Measures (The Blue Team Perspective)

To prevent these attacks in a professional environment, I recommend the following:

1. **DAI (Dynamic ARP Inspection**): A security feature in high-end switches that checks every ARP packet against a trusted database. If it doesn't match, the packet is dropped.
2. **VPN (Virtual Private Network):** Even if an attacker intercepts the packets, they will only see encrypted VPN traffic, making the data useless.
3. **Static ARP Entries:** For critical servers, the MAC address can be manually mapped to the IP in the computer's settings, so it never asks the network "who has this IP?"
4. **Use of Modern Protocols:** Phase out FTP for SFTP, and ensure all web traffic is forced over HTTPS.

7. Conclusion

This project successfully demonstrated the mechanics of a Man-in-the-Middle attack. By mastering both the manual and automated (Bettercap) methods, I have gained a deeper understanding of network layer vulnerabilities. This knowledge is vital for my role as a Red Teamer, as it allows me to better simulate real-world threats and help organizations strengthen their defenses.

Academic Disclaimer: *This project was performed in a closed, authorized lab environment. The techniques described are for educational and professional auditing purposes only. Unauthorized use of these methods is a criminal offense.*